ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO THỰC HÀNH ROS GIỮA KỲ

Giảng viên hướng dẫn: TS. Lê Xuân Lực
Ths. Nguyễn Thiên Hảo
Sinh viên thực hiện: Nguyễn Trọng Quân – 23020760

1 Dạng Robot, động học và kích thước

1.1 1. Dạng Robot

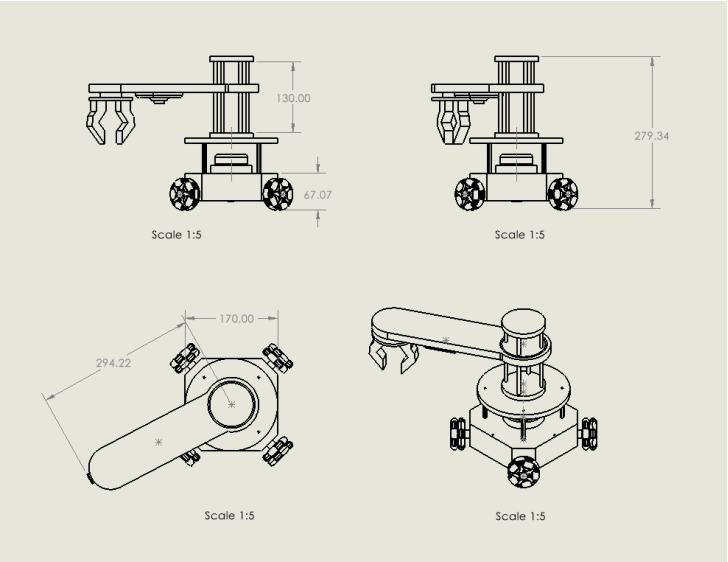
- - Loại di chuyển: 4 bánh omni.
- Khớp 1: Khớp xoay (revolute)
- Khớp 2: Tịnh tiến (prismatic)
- Cảm biến: Lidar, camera, IMU

1.2 Động học

- Di chuyển tiến, lùi, quay trái, quay phải, chéo, quay tại chỗ
- Bậc tự do: 2 bậc

1.3 Kích thước

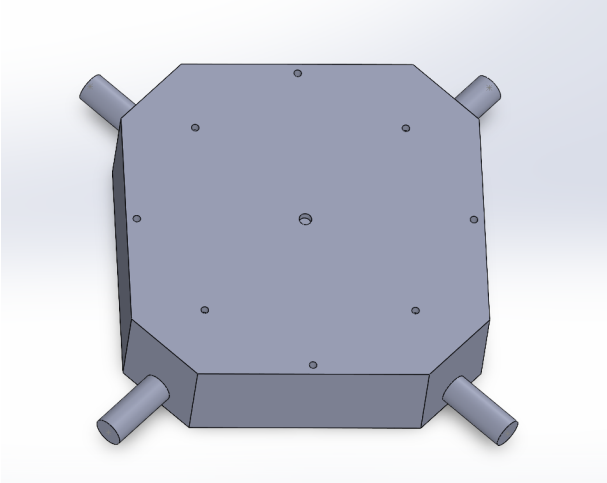
- Thân xe:
  - Chiều dài: 170mm
  - Chiều rộng: 170mm
  - Chiều cao: 50mm
- Chiều dài khớp tịnh tiến: 130mm
- Bán kính tay xoay: 290mm



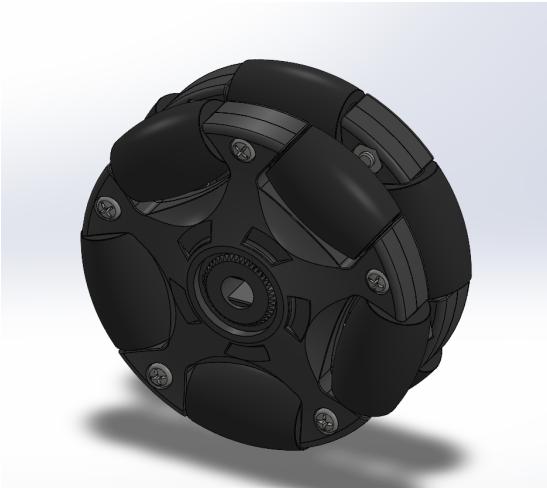
Bản vẽ Robot

2 Thiết kế, đặt trục tọa độ

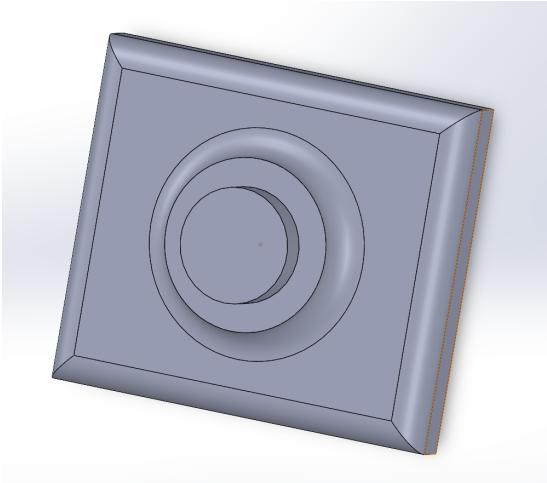
Thiết kế các path bằng Solidworks:



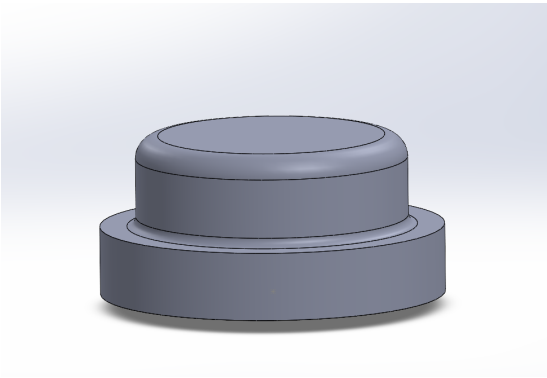
Thân robot



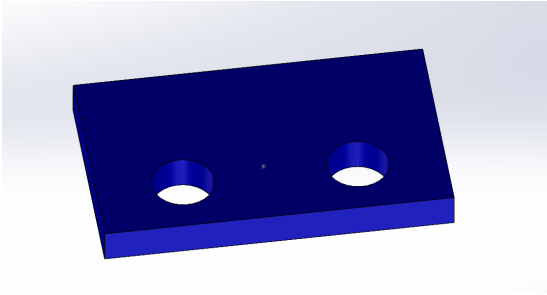
(a) Bánh omni



(b) Camera



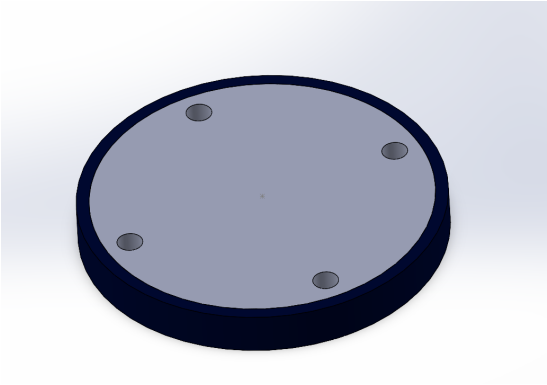
(c) Lidar



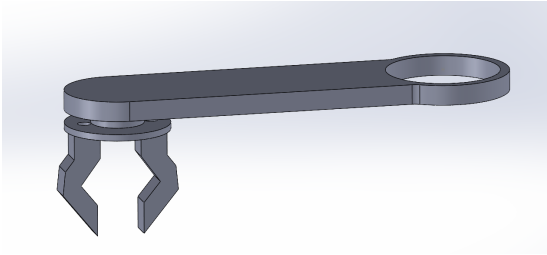
(d) IMU

- Do một phần thời gian không đủ và chi tiết có cấu tạo phức tạp nên em không thể

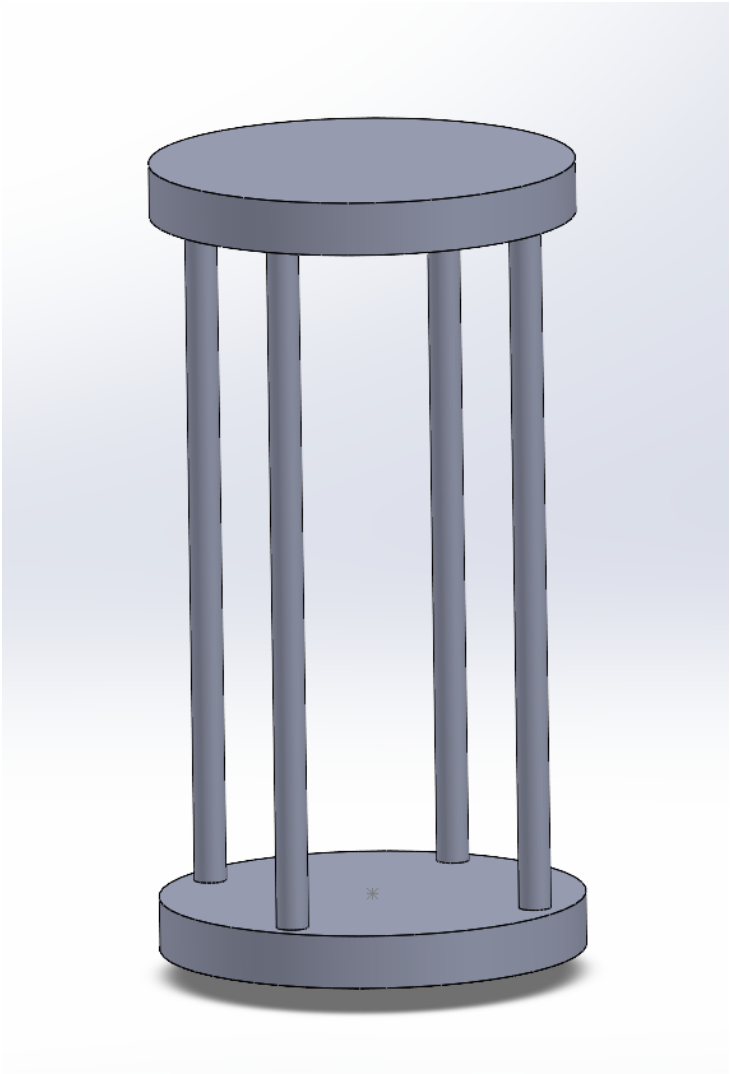
vẽ lại được bánh omni. Vì vậy em đã tham khảo và sử dụng mẫu bánh omni 58mm có sẵn.



(a) Vòng bi hỗ trợ xoay

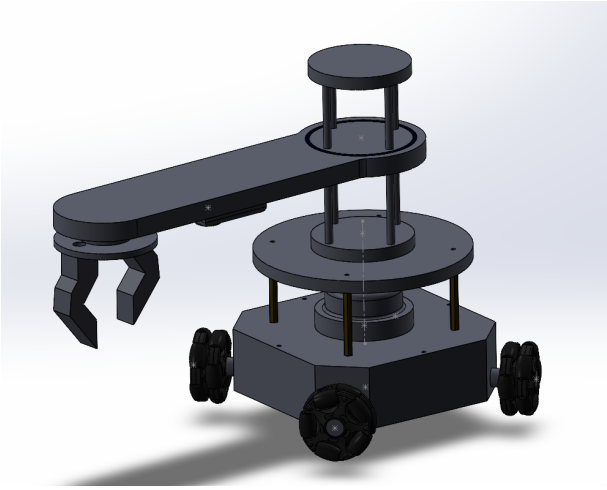


(b) Tay máy



(c) Vít me trục tịnh tiến

Cơ cấu tay máy 2-DOF

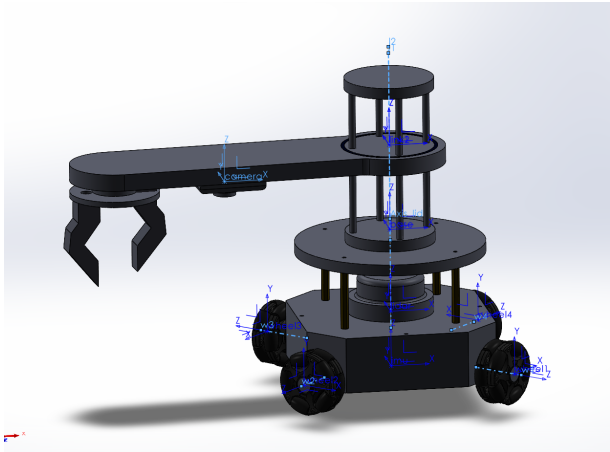


Robot bản hoàn chỉnh

Đặt hệ trục tọa độ:

- Các chi tiết lidar, camera, imu, vòng bi, base được đặt hệ trục tọa độ với chiều x hướng về phía hướng di chuyển phía trước của xe, trục y nằm về phía bên trái và trục z có chiều hướng lên phía trên.
- Mỗi bánh xe có hệ tọa độ riêng với gốc nằm ở tâm của bánh và trục z cùng chiều với trục quay của bánh.
- Các khớp xoay và tịnh tiện cũng được đặt chiều của trục z cùng với chiều chuyển động và chiều xoay.

Robot sau khi hoàn thành đặt trục tọa độ:

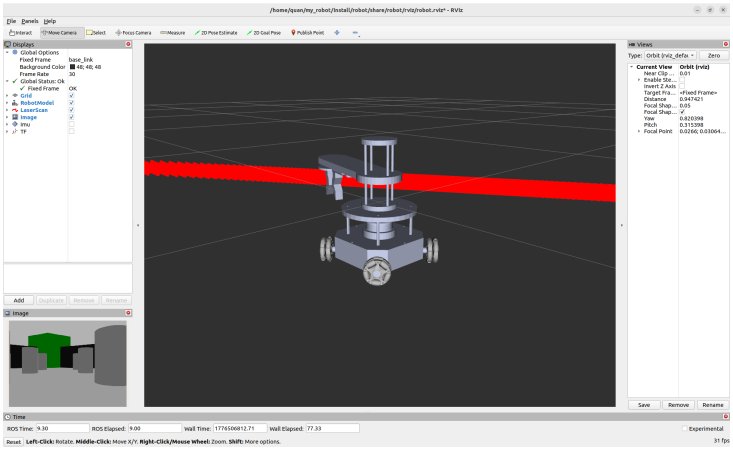


Đặt trục Robot

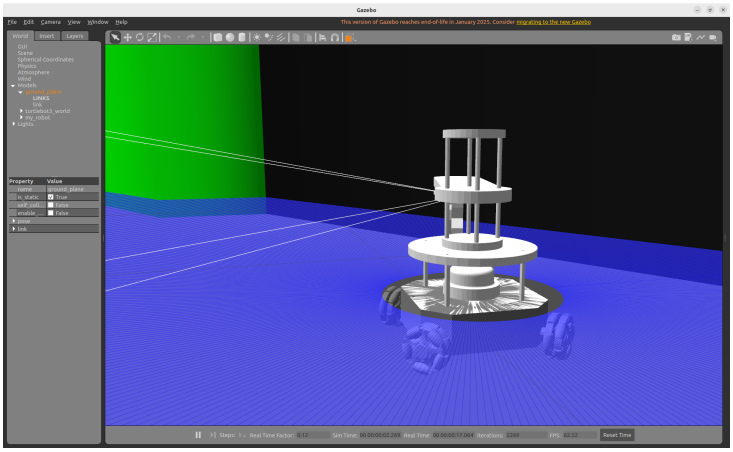
3 Mô tả file urdf, liên kết của các link, các cảm biến, mô tả gazebo

File urdf giúp chúng ta định nghĩa các khớp, các khâu, giúp chúng ta thấy được sự liên kết của các link, khai báo các giá trị của cảm biến và mô tả gazebo.

Robot hiển thị chính xác trong Rviz và Gazebo

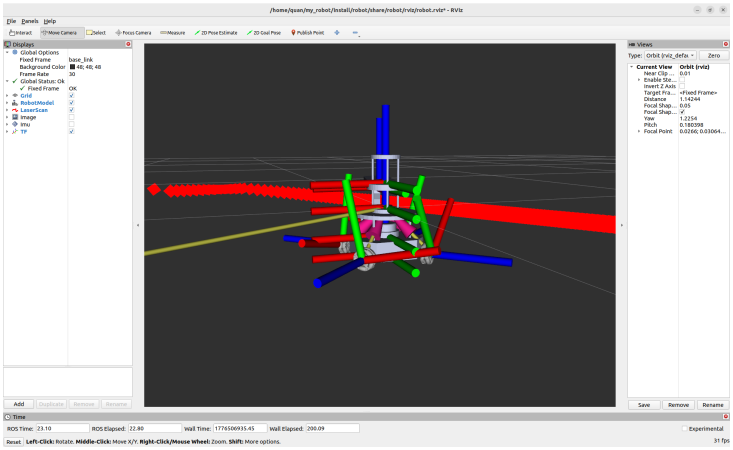


Hình ảnh của Robot trong môi trường Rviz



Hình ảnh của Robot trong môi trường Gazebo

Hệ trục tọa độ hiển thị đúng như trong Solidworks:



Hệ trục tọa độ của xe trong Rviz

Phần code khai báo base_link:

```
<link name="base_link">
    <inertial>
      <origin xyz="-3.24159650053057E-07 -2.77555756156289E-17
         -0.0880190321011237" rpy="0 0 0" />
      <mass value="1.60871892135771" />
      <inertia ixx="0.003820857251224" ixy="-5.45945661495239E-17"
          ixz="-1.03129716330073E-08" iyy="0.0038208610045686" iyz
         ="7.20214372703818E-20" izz="0.0070632727871881" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/base_link.STL" />
      </geometry>
      <material name="">
        <color rgba="0.792156862745098 0.819607843137255
           0.933333333333333 1" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/base_link.STL" />
      </geometry>
    </collision>
  </link>
```

base_link là tọa độ gốc của toàn bộ robot. Khối lượng được định nghĩa chính xác là ≈ 1.61 kg. Trọng tâm (origin xyz) được hạ thấp xuống trục âm của Z ($-0.088m$) giúp thiết lập độ cân bằng tĩnh cho khung gầm, ngăn ngừa lật khi cánh tay phía trên hoạt động. Ma trận quán tính (Inertia Matrix) được khai báo đầy đủ để engine vật lý (Gazebo) tính toán mô-men xoắn.

Phần code khai báo bánh xe:

```
<link name="wheel1">
    <inertial>
      <origin xyz="9.15333442930155E-08 1.59477313360634E-06 -0.0131999999356674" rpy="0 0 0" />
      <mass value="0.042746824244381" />
      <inertia ixx="3.36135178077625E-06" ixy="1.54604789471726E-10" ixz="2.77423227933973E-09" iyy="3.36403756201209E-06" iyz="-1.59203332727708E-10" izz="6.22807467011582E-06" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/wheel1.STL" />
      </geometry>
      <material name="">
        <color rgba="0.752941176470588 0.752941176470588 0.752941176470588 1" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/wheel1.STL" />
      </geometry>
    </collision>
  </link>
  <joint name="w1" type="continuous">
    <origin xyz="0.0947487373415291 -0.0947487373415291 -0.113000000000001" rpy="1.5707963267949 0 0.785398163397449" />
    <parent link="base_link" />
    <child link="wheel1" />
    <axis xyz="0 0 1" />
    <limit effort="100.0" velocity="20.0" /> <dynamics damping="0.1" friction="0.1" />
  </joint>
```

Việc khai báo loại khớp "type = continues " cho phép bánh xe quay vô hạn quanh trục z. Bên cạnh đó, thẻ dynamics giúp cấp độ rít cơ khí (damping) và ma sát tĩnh (friction) nhỏ để ổn định chuyển động quay, tránh hiện tượng trôi bánh (drifting) khi dừng lệnh.

Code khai bảo cảm biến lidar trong gazebo:

```
<link name="lidar">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.12" />
      <inertia ixx="1e-4" ixy="0" ixz="0" iyy="1e-4" iyz="0" izz="1e-4" />
    </inertial>
```

```
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/lidar.STL" />
      </geometry>
      <material name="">
        <color rgba="0.792156862745098 0.819607843137255
           0.933333333333333 1" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/lidar.STL" />
      </geometry>
    </collision>
  </link>
  <joint name="lid" type="fixed">
    <origin xyz="0 0 -0.0750000000000013" rpy="0 0 0" />
    <parent link="base_link" />
    <child link="lidar" />
    <axis xyz="0 0 0" />
  </joint>

<gazebo reference="lidar">
    <sensor name="lidar_sensor" type="ray">
      <pose>0 0 0 0 0 0</pose>
      <always_on>true</always_on>
      <visualize>true</visualize>
      <update_rate>10</update_rate>
      <ray>
        <scan>
          <horizontal>
            <samples>360</samples>
            <resolution>1.0</resolution>
            <min_angle>-3.14159</min_angle>
            <max_angle>3.14159</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.12</min>
          <max>12.0</max>
          <resolution>0.015</resolution>
        </range>
      </ray>
      <plugin name="gazebo_ros_lidar" filename="
         libgazebo_ros_ray_sensor.so">
        <ros>
          <remapping>~/out:=scan</remapping>
        </ros>
        <output_type>sensor_msgs/LaserScan</output_type>
```

```
        <frame_name>lidar</frame_name>
      </plugin>
    </sensor>
  </gazebo>
```

- Lidar được gắn cố định vào thân chính của robot(base_link).

- Ở đây em khai báo lidar có các thông số cảm biến quang học tia (Ray sensor). Quét với tần số 10Hz, lấy 360 điểm lấy mẫu cho một vòng tròn hoàn chỉnh (360°). Điểm mù là $0.12m$ và cự ly quét tối đa $12m$. Plugin gazebo_ros_lidar chịu trách nhiệm chuyển đổi dữ liệu vật lý này thành bản tin sensor_msgs/LaserScan xuất ra topic /scan trong ROS 2.

Khai báo camera trong gazebo:

```
<link name="camera">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.05" />
      <inertia ixx="1e-4" ixy="0" ixz="0" iyy="1e-4" iyz="0" izz="
         1e-4" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/camera.STL" />
      </geometry>
      <material name="">
        <color rgba="0.792156862745098 0.819607843137255
           0.933333333333333 1" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/camera.STL" />
      </geometry>
    </collision>
  </link>
  <joint name="cam" type="fixed">
    <origin xyz="-0.035361448648695 3 -0.145772315442493
       -0.0234999999999997" rpy="0 0 0" />
    <parent link="link2" />
    <child link="camera" />
    <axis xyz="0 0 0" />
  </joint>

<gazebo reference="camera">
    <sensor name="camera_sensor" type="camera">
      <pose>0 0 0 0 0 0</pose>
      <always_on>true</always_on>
      <visualize>true</visualize>
```

```
      <update_rate>30.0</update_rate>
      <camera>
        <horizontal_fov>1.047198</horizontal_fov>
        <image>
          <width>640</width>
          <height>480</height>
          <format>R8G8B8</format>
        </image>
        <clip>
          <near>0.05</near>
          <far>8.0</far>
        </clip>
      </camera>
      <plugin name="gazebo_ros_camera" filename="
         libgazebo_ros_camera.so">
        <ros>
          <remapping>~/image_raw:=camera/image_raw</remapping>
        </ros>
        <camera_name>robot_camera</camera_name>
        <frame_name>camera</frame_name>
      </plugin>
    </sensor>
  </gazebo>
```

- Lidar được gắn cố định vào thân chính của robot(base_link) thông qua joint_cam

-Camera được khai báo có độ phân giải hình ảnh 640 × 480 pixel, định dạng màu R8G8B8, góc nhìn ngang (FOV) là 1.047 rad (≈ 60°). Xuất dữ liệu qua topic /camera/image_raw với tần số 30Hz.

Khai báo cảm biến imu trong gazebo:

```
<link name="imu">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.01" />
      <inertia ixx="1e-5" ixy="0" ixz="0" iyy="1e-5" iyz="0" izz="
         1e-5" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://robot/meshes/imu.STL" />
      </geometry>
      <material name="">
        <color rgba="0.792156862745098 0.819607843137255
           0.933333333333333 1" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
```

```
        <mesh filename="package://robot/meshes/imu.STL" />
      </geometry>
    </collision>
  </link>
  <joint name="i1" type="fixed">
    <origin xyz="0 0 -0.125000000000001" rpy="0 0 0" />
    <parent link="base_link" />
    <child link="imu" />
    <axis xyz="0 0 0" />
  </joint>

<gazebo reference="imu">
    <sensor name="imu_sensor" type="imu">
      <always_on>true</always_on>
      <update_rate>100</update_rate>
      <visualize>true</visualize>
      <plugin name="gazebo_ros_imu" filename="
         libgazebo_ros_imu_sensor.so">
        <ros>
          <remapping>~/out:=imu/data</remapping>
        </ros>
        <initial_orientation_as_reference>false</
           initial_orientation_as_reference>
        <frame_name>imu</frame_name>
      </plugin>
    </sensor>
  </gazebo>
```

-Khai báo tần số trích xuất cao (100Hz) qua plugin gazebo_ros_imu_sensor để theo dõi gia tốc tuyến tính và vận tốc góc

Khai báo cho tay máy:

```

<joint name="l1" type="revolute">
    <origin xyz="0 0 0.065585" rpy="0 0 0" />
    <axis xyz="0 0 1" />
    <limit lower="-3.14" upper="3.14" effort="100.0" velocity="
       3.14" />
    <dynamics damping="10.0" friction="5.0" />
  </joint>

  <joint name="l2" type="prismatic">
    <limit lower="-0.05" upper="0.7" effort="100.0" velocity="1.0"
        />
    <dynamics damping="20.0" friction="10.0" />
  </joint>
```

- Khớp l1 (Revolute): Cung cấp chuyển động xoay quanh trục Z, giới hạn trong vòng 360° (±3.14 rad)

- Khớp l2 (Prismatic): Cung cấp chuyển động tịnh tiến dọc trục Z, được giới hạn trong khoảng $-0.05m$ đến $0.7m$.
- Để cánh tay hoạt động an toàn và chống rơi tự do do trọng lực, cả hai khớp đều được trang bị lực hãm (Damping từ $10 - 20$) và ma sát cơ khí (Friction từ $5 - 10$) rất lớn. Đồng thời khối lượng link1 (0.04 kg) và link2 (0.08 kg) được thiết kế siêu nhẹ để đảm bảo tỷ lệ khối lượng (Mass Ratio) an toàn cho môi trường mô phỏng.

3.1 Cấu trúc cây liên kết của robot

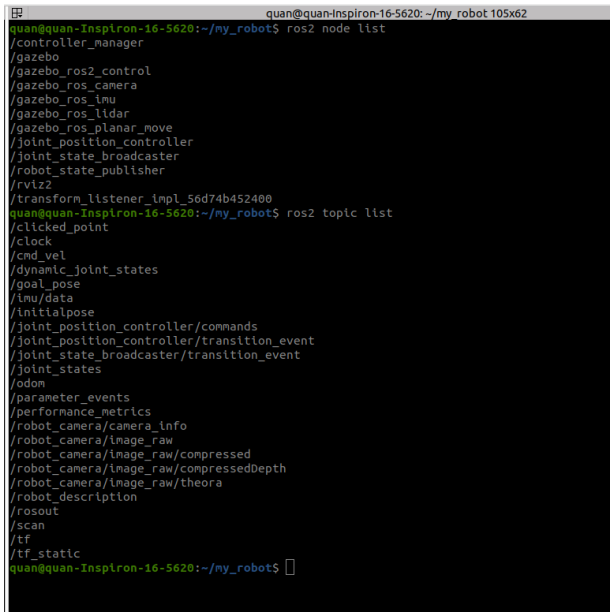
Toàn bộ cấu trúc theo dạng cây phân nhánh (tree structure), lấy base_link làm gốc:

-[**base_link**]

- → (Khớp Continuous w1, w2, w3, w4) → [Bánh xe 1, 2, 3, 4]
- → (Khớp Fixed lid, i1) → [lidar], [imu]
- → (Khớp Revolute l1) → [link1] (Đế tay máy)
  - → (Khớp Prismatic l2) → [link2] (Trục tịnh tiến)
    - → (Khớp Fixed cam) → [camera]

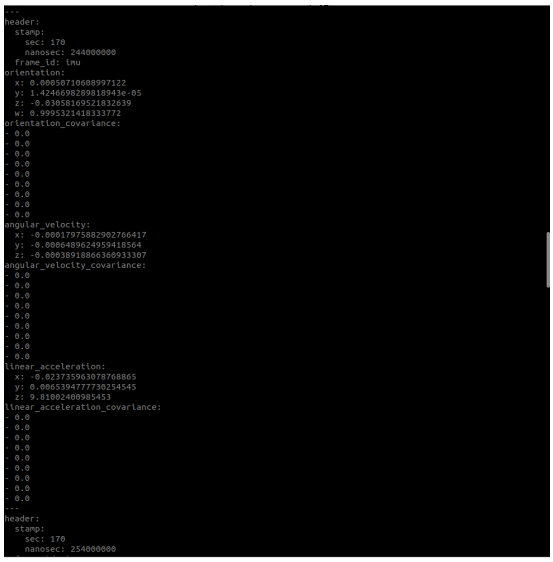
Cấu trúc này đảm bảo rằng khi base_link di chuyển, toàn bộ bánh xe và cảm biến thân dưới sẽ đi theo. Khi link1 xoay, link2 và camera sẽ xoay theo; và khi link2 nâng lên, camera cũng được nâng lên theo.

Sau các bước khai báo cấc link trong file urdf trên và khởi chạy môi trường mô phỏng, hệ thống đã ghi nhận thành công sự xuất hiện của các node và các topics tương ứng. Việc kiểm tra thông qua công cụ dòng lệnh ros2 topic list cho thấy các dữ liệu cảm biến đang được publish ổn định, đáp ứng đúng yêu cầu thiết kế

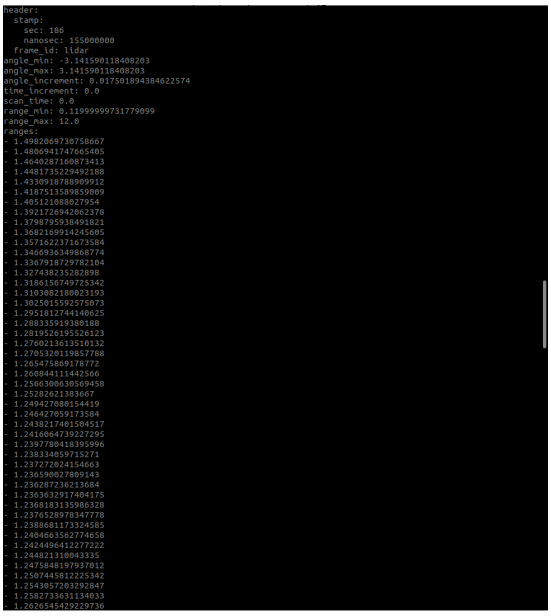


Các topic và node của hệ thống

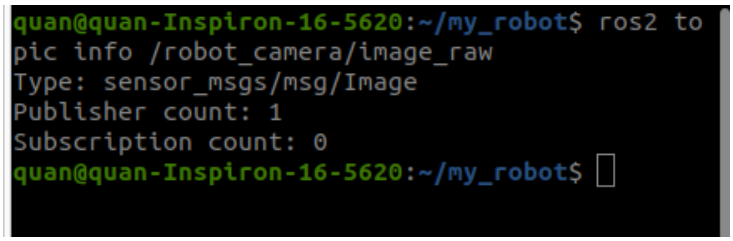
Dữ liệu các cảm biến trả về:



(a) Dữ liệu imu



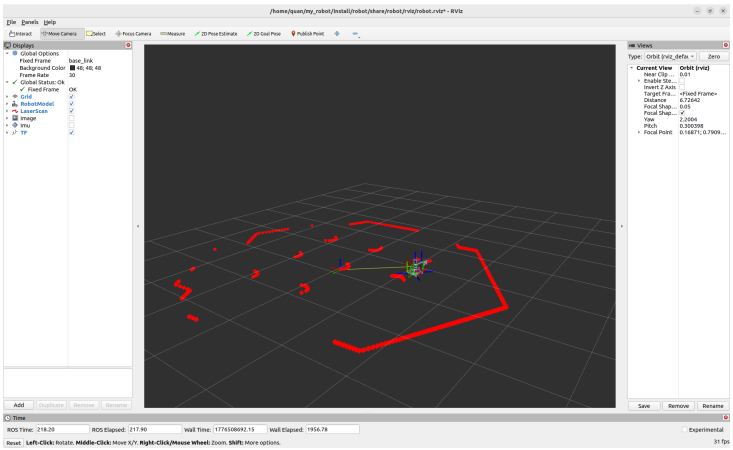
(b) Dữ liệu lidar



(c) Dữ liệu camera trả về

Dữ liệu hệ thống cảm biến trên Robot

Dữ liệu cảm biến hiển thị trong Rviz:



Các cảm biến hoạt động trong Rviz

3.2 Khai báo và thiết lập thông số cho gazebo

```
<gazebo reference="wheel1">
    <mu1>1.0</mu1>
    <mu2>0.0</mu2>
    <kp>1000000.0</kp>
    <kd>100.0</kd>
  </gazebo>
```

Khai báo thiết lập cho robot omni trong môi trường gazebo:

- mu1 = 1.0: Độ bám dính hướng tiến (Lực ma sát dọc) rất cao để xe có thể tạo lực đẩy.
- mu2 = 0.0: Độ bám dính trượt ngang bị triệt tiêu hoàn toàn. Điều này giả lập đặc tính vật lý của các con lăn trên bánh Mecanum/Omni thực tế, cho phép bánh xe lướt ngang mà không bị khóa ma sát.
- kp, kd: Hệ số độ cứng và độ đàn hồi bề mặt tiếp xúc, giữ cho bánh xe không bị lún qua mặt sàn Gazebo.

3.3 Mô tả cơ chế điều khiển trên gazebo

Điều khiển robot trong môi trường Gazebo được vận hành thông qua một hệ thống phân tầng, kết hợp giữa các Plugin mô phỏng và khung làm việc ros2_control.

Đầu tiên là robot.urdf (Lớp mô tả vật lý và Plugin): Có vai trò khai báo các thông số và plugin điều khiển cho robot. Nó không chỉ định nghĩa hình dáng mà còn chứa các thẻ <gazebo> để tích hợp các trình điều khiển phần cứng ảo (libgazebo_ros2_control.so và libgazebo_ros_planar_move.so).

Sau đó là file controllers.yaml là file được tạo ra để cấu hình thuật toán. File này giúp định nghĩa các bộ điều khiển cụ thể sẽ được chạy. Ví dụ: joint_state_broadcaster (để đọc trạng thái khớp) và joint_position_controller (để điều khiển vị trí tay máy). Tệp này quy định các khớp (l1, l2) sẽ thuộc quyền quản lý của bộ điều khiển nào.

Xe được điều khiển bằng plugin gazebo_ros_planar_move. Nó sẽ truy cập topic /cmd_vel khi người dùng gửi lệnh Twist (vx, vy, yaw)

Tay máy được điều khiển bằng ros2_control sau khi gửi lệnh dưới dạng (góc xoay, vị trí khớp tịnh tiến) dữ liệu sẽ được truyền tới topic /joint_position_controller/commands. Sau đó controller_manager nhận dữ liệu, đối chiếu với các giới hạn (limit) trong URDF, sau đó truyền tín hiệu điều khiển tới các khớp l1 và l2 trong Gazebo thông qua plugin GazeboSystem.

Cuối cùng là 1 file đóng vai trò là tổng quản, file display.launch.py. Khi chạy file này thì cả hệ thống sẽ được khởi động.

- Nạp URDF vào Gazebo.

- Kích hoạt controller_manager để đọc tệp .yaml
- Chạy các "spawner" để kích hoạt các bộ điều khiển cụ thể.

3.3.1 Điều khiển Robot di chuyển

Điều khiển xe di chuyển tiến, lùi, rẽ trái, phải và xoay trái , phải bằng ros2 run teleop_twist_keyboard teleop_twist_keyboard

(a) Tiến lên phía trước

(b) Đi về phía bên phải

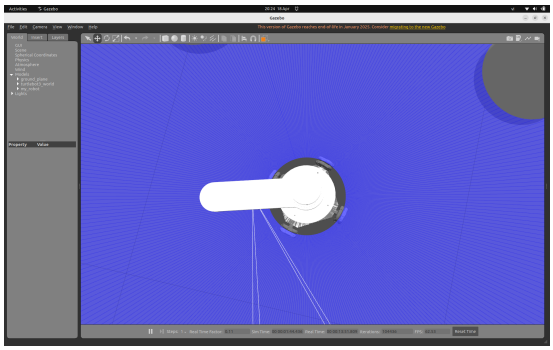
(c) Xoay tại chỗ về bên trái

(d) Đi chéo

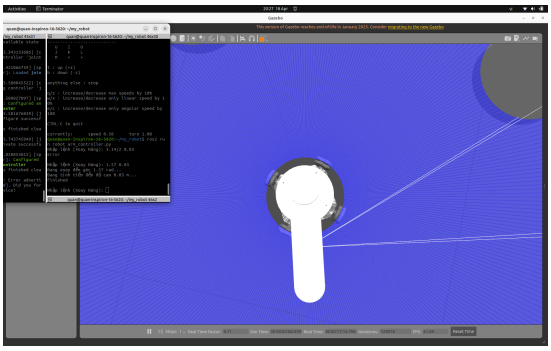
Dữ liệu của topic /cmd_vel

3.3.2 Điều khiển tay máy

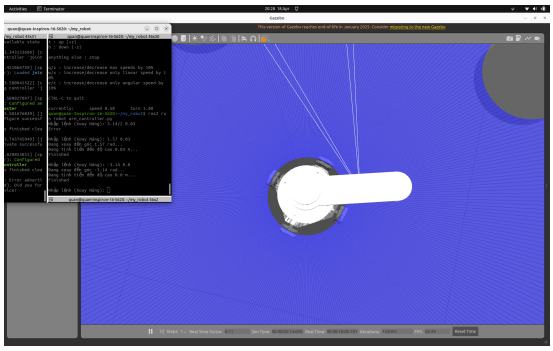
Tay máy được điều khiển bằng node arm_controller.py thực hiện chu trình kiểm tra trạng thái hiện tại sau đó người dùng sẽ tiến hành nhập góc quay và độ cao cần đến. Robot sẽ tiến hành quay đến góc cần đến sau đó mới tiến hành tịnh tiến độ cao của tay máy.



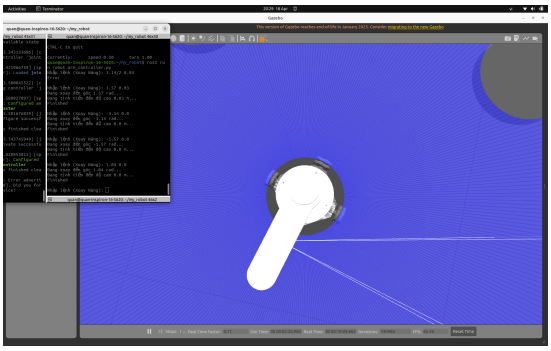
(a) Tay máy ở trạng thái spawn



(b) Tay máy khi xoay đến vị trí pi/2 và độ cao 0.03m



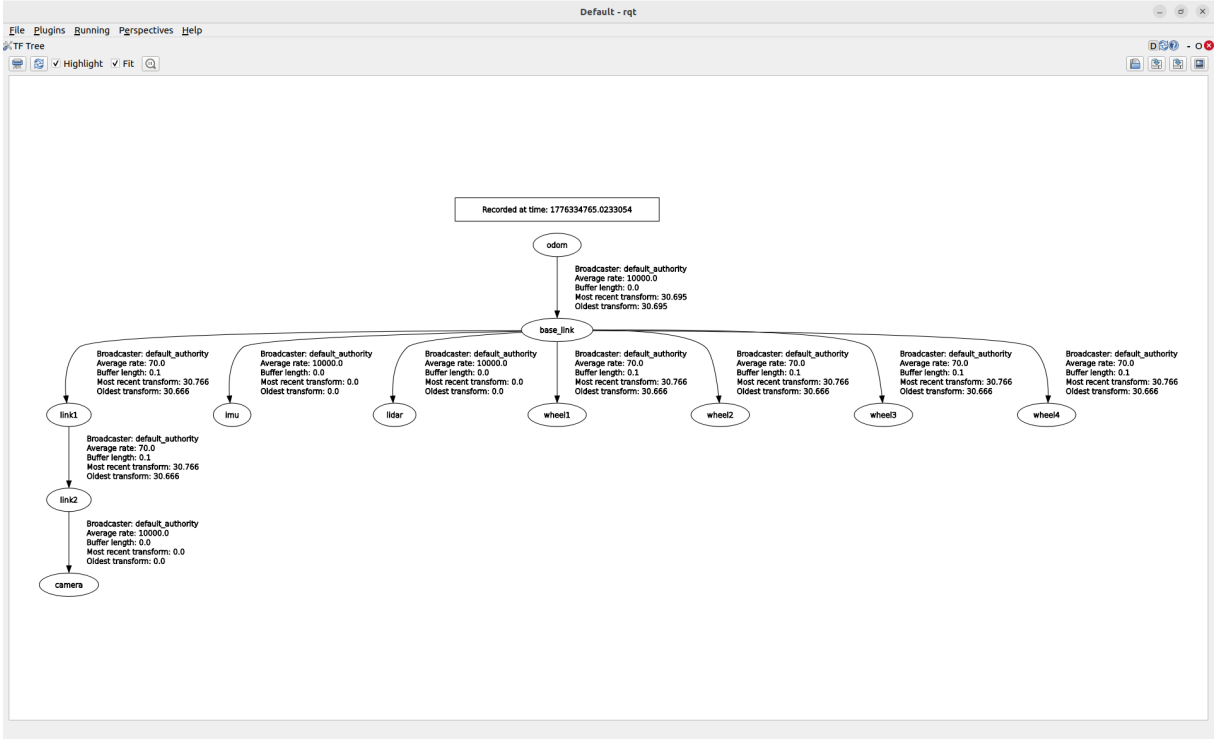
(c) Xoay đến vị trí trừ pi và độ cao 0.0m



(d) Góc quay pi/3 và độ cao 0.0m

Tay robot quay trong môi trường gazebo

Cây TF_tree:



Cây TF

4 Các thành phần chính của code, cấu trúc của dự án

Cấu trúc của dự án:

```
4_banh_omni
├── CMakeLists.txt              #File xây dựng package
├── config                      # Chứa file cấu hình và Rviz
│   └── controllers.yaml
├── launch                      # Chứa file launch (mở gazebo, khởi tạo môi trường Waffe,...)
│   └── display.launch.py
├── meshes                      # Chứa file STL 3D
│   ├── banh_xe_1.STL
│   ├── banh_xe_2.STL
│   ├── banh_xe_3.STL
│   ├── banh_xe_4.STL
│   ├── base_link.STL
│   ├── camera.STL
│   ├── imu.STL
│   ├── khop_tinh_tien.STL
│   ├── lidar.STL
│   ├── link1.STL
│   ├── link2.STL
│   ├── tay_robot.STL
│   ├── wheel1.STL
│   ├── wheel2.STL
│   ├── wheel3.STL
│   └── wheel4.STL
├── package.xml                 # File mô tả package ROS2
├── rviz                        # Chứa file lưu cấu hình khởi động Rviz
│   └── robot.rviz
├── scripts                     # File Scripts điều khiển tay máy
│   └── arm_controller.py
└── urdf                        # File urdf khai báo thông tin robot
    ├── robot.csv
    └── robot.urdf
```

Robot 4 bánh omni

-Các thành phần chính

controllers.yaml

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100
    use_sim_time: true

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    joint_position_controller:
      type: position_controllers/JointGroupPositionController

    wheel_velocity_controller:
      type: velocity_controllers/JointGroupVelocityController


# Khai bao danh sach khop
joint_position_controller:
```

```
  ros__parameters:
    joints:
      - l1
      - l2

wheel_velocity_controller:
  ros__parameters:
    joints:
      - w1
      - w2
      - w3
      - w4
```

display.launch.py

```
import os
import xacro
from ament_index_python.packages import
   get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription,
   SetEnvironmentVariable
from launch.launch_description_sources import
   PythonLaunchDescriptionSource

def generate_launch_description():
    # 1. Khai bao cac duong dan
    pkg_share = get_package_share_directory('robot')

    # Lay duong dan toi package turtlebot3_gazebo
    pkg_tb3_gazebo = get_package_share_directory('
       turtlebot3_gazebo')

    # Chon map Waffle
    world_path = os.path.join(pkg_tb3_gazebo, 'worlds', '
       turtlebot3_world.world')

    # Duong dan den workspace_models
    workspace_models_path = os.path.join(pkg_share, '..')

    # 2. Xu ly file URDF
    xacro_file = os.path.join(pkg_share, 'urdf', 'robot.urdf')
    robot_desc = xacro.process_file(xacro_file).toxml()

    # 3. Thiet lap bien moi truong GAZEBO_MODEL_PATH
    set_gazebo_model_path = SetEnvironmentVariable(
        name='GAZEBO_MODEL_PATH',
        value=[
            os.environ.get('GAZEBO_MODEL_PATH', ''),
            ':', workspace_models_path,
            ':', os.path.join(pkg_tb3_gazebo, 'models')
```

```
        ]
    )

    # 4. Node Robot State Publisher
    robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': robot_desc, '
           use_sim_time': True}]
    )

    # 5. Mo Gazebo va Nap Map
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(get_package_share_directory('gazebo_ros')
               , 'launch', 'gazebo.launch.py')
        ),
        launch_arguments={'world': world_path}.items()
    )

    # 6. Node Spawn Robot
    # Chinh toa do spawn
    spawn_entity = Node(
        package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=[
            '-topic', 'robot_description',
            '-entity', 'my_robot',
            '-x', '-2.0', '-y', '-0.5', '-z', '0.01'
        ],
        output='screen'
    )

    # Khai bao duong dan toi file cau hinh RViz cua ban
    rviz_config_path = os.path.join(pkg_share, 'rviz', 'robot.rviz
       ')

    # Node RViz2
    rviz2 = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        # Them arguments '-d' de nap file cau hinh
        arguments=['-d', rviz_config_path],
        parameters=[{'use_sim_time': True}]
    )

    # 8. Controller Spawners
    load_joint_state_broadcaster = Node(
        package="controller_manager",
```

```
        executable="spawner",
        arguments=["joint_state_broadcaster"],
    )

    load_joint_position_controller = Node(
        package="controller_manager",
        executable="spawner",
        arguments=["joint_position_controller"],
    )

    return LaunchDescription([
        set_gazebo_model_path,
        robot_state_publisher,
        gazebo,
        spawn_entity,
        rviz2,
        load_joint_state_broadcaster,
        load_joint_position_controller,
    ])
```

arm_controller.py

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray
import time

def main(args=None):
    rclpy.init(args=args)
    node = rclpy.create_node('arm_sequential_controller')

    publisher = node.create_publisher(
        Float64MultiArray,
        '/joint_position_controller/commands',
        10)

    # trang thai hien tai cua robot
    current_angle = 0.0
    current_height = 0.0

    try:
        while rclpy.ok():
            user_input = input("Nhap lenh (Xoay Nang): ")

            if user_input.lower() == 'q':
                break

            parts = user_input.split()
            if len(parts) != 2:
                print("Error\n")
                continue
```

```python

            try:
                target_angle = float(parts[0])
                target_height = float(parts[1])

                # Xoay
                print(f"Dang xoay den goc {target_angle} rad...")
                msg = Float64MultiArray()
                msg.data = [target_angle, current_height] # Gui goc moi, do cao cu
                publisher.publish(msg)

                # Tam dung chuong trinh de cho mo to trong Gazebo xoay xong
                time.sleep(2.0)

                # Dieu khien tinh tien
                print(f"Dang tinh tien den do cao {target_height} m...")
                msg.data = [target_angle, target_height] # Gui goc MOI, do cao MOI
                publisher.publish(msg)

                # Tam dung 2 giay cho nang xong
                time.sleep(2.0)

                print("Finished\n")

                # Cap nhat vi tri
                current_angle = target_angle
                current_height = target_height

            except ValueError:
                print("Error\n")

    except KeyboardInterrupt:
        node.get_logger().info('shutting down arm controller node...')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

5 Kết luận

Báo cáo đã thể hiện toàn vẹn dự án thiết kế và mô phỏng Robot 4 bánh omni có tích hợp các cảm biến lidar, imu, camera và tay máy 2 bậc tự do thiết kế kiểu vít me.

- Thiết kế cơ khí và hệ thống: Đã hiển thị toàn vẹn bản vẽ 3D robot trong solidworks trong gazebo và rviz mà không gây ra lỗi.
- Dữ liệu cảm biến: Các cảm biến đã hoạt động đúng với công dụng và những gì được khai báo trong file URDF, data được trả về đầy đủ.
- Điều khiển hệ thống: Robot và tay máy đều có thể hoạt động bình thường, không gây ra lỗi.

5.1 Những khó khăn trong quá trình làm

Trong quá trình làm dự án, còn gặp một số khó khăn như sau:

- Quá trình đặt trục và đặt tọa độ còn gặp một số vấn đề, khiến cho robot khi được spawn vào gazebo không được hiển thị đầy đủ hoặc hiển thị lỗi.
- Chưa source đủ đường dẫn và môi trường khiến cho bị kẹt, dính lỗi not responding của client gazebo.
- Trong quá trình chỉnh sửa file URDF còn gặp một số khó khăn trong việc tối ưu thông số (Ví dụ như trọng lượng, tọa độ cảm biến, gắn plugin cho robot,...)

Chi tiết toàn bộ dự án được lưu tại đây: Link

Video chạy dự án xem tại đây: Link